

# JAVA INTERVIEW PREPARATION

CHAPTER 36

SPRING TRANSACTION MANAGEMENT

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

# Spring Transaction Management: Boundaries, Propagation, Rollback, and Proxy Reality

Senior / Lead Java Developer Reading Chapter

`@Transactional` is metadata for an interceptor, not a database command embedded in the method. The interceptor chooses a transaction manager, begins or joins a transaction, binds resources to the execution context, invokes the target, then commits or rolls back according to outcome and rules. Correct transaction design begins with the business consistency boundary.

## The Invocation Model

```
caller -> transactional proxy
        -> inspect metadata
        -> begin or join transaction
        -> invoke service method
        -> commit or rollback
        -> release resources
```

```
@Service
class TransferService {
    @Transactional
    public void transfer(AccountId from, AccountId to, Money amount) {
        accounts.debit(from, amount);
        accounts.credit(to, amount);
    }
}
```

Place the boundary around a cohesive application operation. Repository-level annotations alone can produce separate transactions that do not preserve a service-level invariant.

## Imperative Transactions Are Commonly Thread-Bound

PlatformTransactionManager-based transactions commonly bind resources to the current thread. Starting another thread does not inherit that transaction.

```
@Transactional
public void createOrder(Order order) {
    repository.save(order);
    executor.submit(() -> auditRepository.save(event));
}
```

The asynchronous task normally runs without the original transaction and may execute before or after commit. Use an outbox, after-commit event with understood durability, or an independently transactional async operation.

ReactiveTransactionManager uses reactive context rather than ThreadLocal state and requires participation in the same reactive pipeline. Mixing models without understanding the manager produces absent transactions.

## Self-Invocation

In default proxy mode, only calls through the proxy are intercepted.

```

public void importAll(List<Item> items) {
    for (Item item : items) {
        importOne(item); // bypasses proxy
    }
}

@Transactional(propagation = REQUIRES_NEW)
public void importOne(Item item) { }

```

The intended new transaction does not appear. Extract `importOne` to another bean, call through an explicit transactional template, or redesign the batch boundary. Self-injection is usually a maintenance smell.

## Propagation REQUIRED

REQUIRED is the default. It joins an existing transaction or starts one.

```

outer REQUIRED starts physical transaction T1
  inner REQUIRED joins T1
commit occurs at outer boundary

```

Each method has a logical transaction scope, but participating scopes share one physical transaction. If an inner scope marks rollback-only and the outer method attempts to commit, Spring can throw `UnexpectedRollbackException`. This prevents the caller from believing a commit succeeded.

## REQUIRES\_NEW

REQUIRES\_NEW suspends the outer transaction and starts an independent physical transaction.

```

outer T1 suspended
  inner T2 begins -> commits/rolls back independently
outer T1 resumes

```

It needs an additional connection while the outer transaction retains its resources. If every request holds one connection and waits for another, the pool can exhaust. Size and design the pool with nested transactions in mind.

Independent commit also changes semantics: an audit T2 may commit even if business T1 later rolls back. That may be intentional, but it is not atomic with T1.

## NESTED and Savepoints

NESTED typically uses a savepoint inside one physical transaction when supported by the transaction manager and resource.

```

physical transaction T1
  savepoint S
  inner work fails -> roll back to S
  outer work may continue

```

It is not equivalent to REQUIRES\_NEW. Support varies, especially across JPA and database combinations. Verify the actual manager behavior rather than relying only on the annotation name.

## Other Propagation Modes

SUPPORTS joins if present but can run without a transaction. MANDATORY fails if none exists.

NOT\_SUPPORTED suspends an existing transaction and runs without one. NEVER fails when a transaction exists. These modes are tools for explicit contracts, not decoration.

## Rollback Rules

By default, runtime exceptions and Errors trigger rollback; checked exceptions do not. Configure rules when the domain contract differs.

```
@Transactional(rollbackFor = PaymentException.class)
public void settle() throws PaymentException { }
```

Catching an exception inside the method prevents the interceptor from seeing it unless code marks rollback-only or rethrows.

```
try {
    repository.write();
} catch (DataAccessException exception) {
    log.warn("ignored", exception); // transaction may attempt commit
}
```

Do not catch broadly and return success. Translate exceptions while preserving rollback intent and original cause.

## Isolation

Isolation controls which concurrent database phenomena can be observed. Spring's `Isolation.DEFAULT` delegates to the underlying system.

```
READ_COMMITTED prevents dirty reads; non-repeatable reads may occur
REPEATABLE_READ strengthens repeated-row consistency, database-specific details matter
SERIALIZABLE strongest standard isolation, lower concurrency and possible retries
```

MVCC, predicate locking, gap locking, and snapshot behavior differ across databases. Isolation names do not replace database documentation and tests.

Lost-update prevention often requires optimistic versioning, conditional updates, row locks, or serialization of ownership, not merely raising an annotation value.

## readOnly, Timeout, and Manager Selection

`readOnly = true` is a hint and optimization opportunity, not a security boundary guaranteeing no writes on every database.

Transaction timeout limits transaction duration according to manager support. It should align with query, lock, request, and downstream time budgets.

Applications with several transaction managers must select the intended one explicitly. Accidentally using a JPA manager for one resource and another manager for a second does not create a distributed atomic transaction.

## Transactions and Remote Calls

Holding a database transaction open during HTTP or message calls retains connections and locks while network latency is unbounded.

```
begin DB transaction
update rows
call remote service for 3 seconds
commit
```

Prefer local transactions plus an outbox for reliable message publication, sagas or compensation for multi-service workflows, and idempotent consumers. A database transaction cannot roll back an already accepted remote side effect.

## Events and After-Commit Work

Transactional event listeners can run at phases such as after commit. They are useful when work should occur only after local commit, but an in-memory listener is not durable if the process crashes after commit and before handling.

Use an outbox or durable queue when delivery is part of correctness. After-commit callbacks suit best-effort cache invalidation, local metrics, or work whose loss is acceptable or recoverable.

## Testing Transactions

Test rollback, commit, isolation-sensitive races, self-invocation, and propagation using the real proxy and database behavior. A test-level transaction that rolls back after each test can hide missing application transactions because repositories happen to participate in the test transaction.

Use integration tests that commit when verifying constraints, generated values, after-commit behavior, and asynchronous visibility.

## Logical Scope Versus Physical Resource Transaction

Propagation is easiest to reason about by drawing both layers. Each annotated invocation has a logical scope; the transaction manager maps that scope to a physical resource transaction.

```
serviceA REQUIRED logical scope A ----+
repository REQUIRED logical scope B ----+--> one JDBC transaction / connection
serviceC REQUIRED logical scope C ----+
```

Each logical REQUIRED scope can declare rollback-only. If an inner scope catches or translates a failure after the transaction has been marked rollback-only, the outer code may continue normally but cannot make the physical transaction commit. `UnexpectedRollbackException` at the outer boundary is therefore a correctness signal, not a random commit failure.

Do not catch it and report success. Find which inner operation marked rollback-only and decide whether the business operation should fail, use a genuinely independent transaction, or handle the condition before the shared transaction becomes invalid.

## Resource Binding and Cleanup

For an imperative JDBC transaction, the manager obtains a connection, configures transaction state, binds a holder to the current thread, and lets participating data-access code reuse it. At completion it commits or rolls back, restores connection settings as needed, unbinds, and returns the connection to the pool.

```
proxy begins
-> acquire/bind connection
-> repositories discover thread-bound connection
-> flush/write work
-> commit or rollback
-> unbind and return resource
```

Code that opens an unrelated connection manually can operate outside the managed transaction. Similarly, using two independent data sources involves two physical transactions unless a distributed transaction mechanism coordinates them. Transaction synchronization is resource-specific infrastructure; the annotation

does not make arbitrary I/O atomic.

Connection acquisition may be lazy in some stacks, but the transaction's logical duration still includes application work. Long CPU work or remote calls inside the method can hold persistence context and locks even when no query is currently executing.

## Flush Is Not the Same as Commit

In JPA, entity changes accumulate in the persistence context and may be flushed before commit, before certain queries, or explicitly. Flush sends SQL and can reveal constraint violations, but the database transaction can still roll back afterward.

```
entity mutation -> persistence context dirty
flush           -> SQL executed, constraints may fail
commit         -> transaction becomes durable/visible per database rules
```

This distinction explains why an exception can occur before the annotated method returns and why testing only in-memory entity state is insufficient. It also explains why a generated identifier or database trigger value may require flush without implying final commit.

Do not call `save` repeatedly merely to "make it transactional." Managed entities are detected through dirty checking. Repository calls and flush strategy should follow persistence semantics and batching requirements, not annotation folklore.

## Isolation Through Concrete Anomalies

Isolation is clearer through schedules than names.

```
non-repeatable read:
T1 reads price=10
T2 updates price=12 and commits
T1 reads same row and sees 12

phantom:
T1 queries all OPEN orders -> 5 rows
T2 inserts another OPEN order and commits
T1 repeats predicate -> 6 rows

lost update:
T1 reads version/balance
T2 reads same value
both write derived result; one overwrites the other
```

Database MVCC implementations differ in which phenomena each named level prevents. `REPEATABLE_READ` on one database may not match another database's locking and snapshot behavior. Use database-specific documentation and concurrent integration tests.

For lost updates, optimistic locking with a version column makes the update conditional on the version originally read. A zero-row update becomes a conflict that can be retried or returned to the user. Pessimistic locking acquires database locks earlier and can be appropriate when conflicts are frequent and work is short, but increases waiting and deadlock risk.

## Rollback Rules Need Type-Safe Intent

Type-based rules such as `rollbackFor = MyCheckedException.class` are safer than broad name patterns. Pattern matching can accidentally match similarly named nested or versioned exception classes. When rollback and no-rollback rules overlap, the strongest matching rule wins under Spring's rule resolution.

Exception translation should preserve category and cause:

```
catch (SQLException failure) {
    throw new InventoryPersistenceException("inventory update failed", failure);
}
```

If the translated exception is checked and no rollback rule includes it, the default may commit. Decide rollback semantics as part of the application exception contract. Conversely, not every business rejection requires rollback if no mutation occurred or a deliberate status must be committed.

Avoid `rollbackFor = Throwable.class` as an automatic policy. It can roll back for conditions the process should not attempt to recover from and obscures deliberate business semantics.

## TransactionTemplate and Explicit Boundaries

Declarative proxy boundaries fit public application operations. `TransactionTemplate` is useful when one method needs several visibly separate units or when self-invocation would otherwise hide the boundary.

```
for (Item item : items) {
    transactionTemplate.executeWithoutResult(status -> importOne(item));
}
```

The programmatic form couples orchestration to transaction infrastructure but makes begin/commit scope explicit. It can be clearer for batch chunking, conditional rollback, or callbacks. Keep domain rules outside the template where possible.

Do not use a template to create thousands of tiny commits without measuring log, round-trip, and constraint cost. Choose chunk size from atomicity, restartability, lock duration, and throughput.

## Retrying a Transaction Safely

Deadlocks, serialization failures, and optimistic conflicts may be transient. A retry must begin a fresh physical transaction after the failed one has rolled back.

```
attempt 1 -> begin T1 -> conflict -> rollback
backoff
attempt 2 -> begin T2 -> re-read current state -> apply -> commit
```

Retrying inside the already failed transaction is ineffective. Ensure retry advice wraps the transactional invocation or orchestrate attempts explicitly. Recompute from fresh data rather than replaying stale entity instances.

Bound attempts, add jittered backoff where contention warrants it, and classify only known transient failures. The entire attempted operation must be idempotent with respect to remote calls, messages, emails, and payments. An outbox can keep message intent atomic with the local database while publication occurs later.

## Multiple Transaction Managers and Reactive Boundaries

An application can have JDBC, JPA, messaging, and reactive transaction managers. Qualify the manager deliberately. One method annotated against manager A does not automatically include resource B.

```
JPA transaction commits database A
Kafka transaction publishes record B
without a supported coordination pattern, outcomes can diverge
```

Chained local commits are not the same as atomic two-phase commit; a process failure between commits creates partial success. Prefer outbox, idempotent consumption, and reconciliation unless distributed atomicity is truly required and supported.

Reactive transaction context travels through the reactive chain. Blocking, subscribing manually, or switching to work outside the propagated context can escape it. Imperative thread-local assumptions and reactive context semantics should not be mixed casually.

## Observability of Transaction Boundaries

Record transaction duration, connection acquisition wait, active/idle pool counts, commit/rollback outcome, deadlock and serialization failures, rows affected, and long-lock evidence. Avoid logging every SQL parameter or customer value.

Separate connection wait from time executing statements and time holding the transaction open after the last query. A method can execute 20 ms of SQL but hold its connection and locks for three seconds during remote I/O.

For an incident, correlate application transaction ID or request trace with database session, lock graph, and slow-query evidence. The Spring proxy can explain the intended boundary; the database proves the actual physical transaction and contention.

## A Strong Interview Explanation

`@Transactional` is metadata interpreted by an interceptor. An external call crosses the proxy, the selected transaction manager begins or joins a physical resource transaction, resources are bound to the execution context, and commit or rollback occurs after the target returns or throws. Self-invocation bypasses that boundary in proxy mode.

Then distinguish propagation precisely: `REQUIRED` creates logical scopes sharing one physical transaction; an inner rollback-only marker can produce `UnexpectedRollbackException`. `REQUIRES_NEW` suspends the outer transaction and needs an additional independent resource. `NESTED` normally uses savepoints where supported. Explain default unchecked rollback, database-specific isolation, flush versus commit, thread-bound imperative context, and why retries need a new transaction. Finish with short local boundaries, outbox/saga for remote consistency, and metrics for connection wait, duration, rollback, and locks.

## Production Perspective

Monitor active connections, acquisition wait, transaction duration, rollbacks, lock waits, deadlocks, and slow queries. Long transactions increase conflict and prevent cleanup of database versions.

`UnexpectedRollbackException` indicates an inner participant marked the shared transaction rollback-only.

Connection-pool exhaustion during `REQUIRES_NEW` points to nested resource demand.

`LazyInitializationException` often means persistence access escaped the intended transaction boundary, not that every association should become eager.

## Interview-Ready Summary

Spring declarative transactions normally use AOP proxies. `REQUIRED` joins or creates a transaction, `REQUIRES_NEW` creates an independent physical transaction while suspending the outer one, and `NESTED` commonly uses savepoints when supported. Self-invocation bypasses proxy advice.

Rollback defaults favor unchecked exceptions. Imperative transactions are commonly thread-bound and do not propagate to new threads. Keep transactions short, avoid remote calls inside them, choose isolation from database behavior, and use outbox or saga patterns for cross-service consistency.



## Revision Notes

- `@Transactional` is metadata interpreted by transactional infrastructure.
- Put transaction boundaries around cohesive application operations.
- Imperative transactions commonly bind resources to the current thread.
- New threads do not inherit the original transaction.
- Self-invocation bypasses proxy transaction advice.
- `REQUIRED` joins an existing transaction or starts one.
- Participating logical scopes share one physical transaction.
- Rollback-only participation can cause `UnexpectedRollbackException`.
- `REQUIRES_NEW` suspends outer work and needs independent resources.
- Nested new transactions can exhaust connection pools.
- `NESTED` commonly uses savepoints when supported.
- Default rollback applies to `RuntimeException` and `Error`.
- Catching an exception can prevent interceptor rollback rules from seeing it.
- Isolation semantics depend on the actual database.
- `readOnly` is generally a hint, not an absolute write prohibition.
- Multiple managers do not automatically create distributed atomicity.
- Avoid remote calls while holding database transactions.
- After-commit listeners are not automatically durable.
- Use an outbox when publication must survive process failure.
- Test actual commit, rollback, proxy, and database behavior.
- Logical `REQUIRED` scopes can independently mark one physical transaction rollback-only.
- Managed repositories reuse resources bound by the transaction manager.
- JPA flush sends SQL but does not make the transaction committed.
- Isolation should be reasoned about through database-specific concurrent schedules.
- Optimistic locking turns stale updates into detectable conflicts.
- Type-based rollback rules are safer than broad exception-name patterns.
- `TransactionTemplate` can make repeated or conditional boundaries explicit.
- A retry must execute in a fresh transaction and re-read current state.
- Multiple local transaction managers do not imply atomic cross-resource commit.
- Observe connection wait, transaction hold time, outcome, and database locks separately.